

Instituto Tecnológico de Costa Rica
Escuela de Computadores
CE 1106: Paradigmas de Programación

Tarea de programación

Aplicación en lenguaje

ensamblador 8086

Integrantes:

Gabriel Soto López - 2024178797

Luis Castro Montenegro - 2013083843

Saddy Guzman Rojas - 2023088184

Profesor

Marco Rivera Meneses

Septiembre, 2025

Índice

1	INTRODUCCIÓN	3
2	DESCRIPCION DE LOS ALGORITMOS DE SOLUCION	3
2.1	Algoritmo de insercion y parseo de datos	4
2.2	Algoritmo de busqueda por indice	4
2.3	Algoritmo de ordenamiento - bubblesort.....	5
2.4	Algoritmo de calculo de estadisticas	6
3	DESCRIPCIÓN DE LOS PROCEDIMIENTOS Y FUNCIONES	7
3.1	Nucleo del programa y gestion de menu	7
3.2	Procesamiento y tokenizacion de entrada.....	8
3.3	Manipulacion de estructuras y conversion de datos.....	10
3.4	Salida, visualizacion y logica de busqueda.....	11
4	DESCRIPCIÓN DE LAS ESTRUCTURAS DE DATOS	13
5	PROBLEMAS ENCONTRADOS	14
6	Plan de Actividades Realizadas.....	15
7	CONCLUSIONES	16
8	RECOMENDACIONES	17
9	BIBLIOGRAFIA	17
10	BITACORA.....	18
	Enlace al repositorio de Github	22

1. Introducción

En este documento se tratará la documentación técnica del sistema Registro CE el cual consiste en una aplicación desarrollada en lenguaje ensamblador 8086, el objetivo principal es diseñar e implementar un sistema interactivo de gestión de notas de estudiantes el cual permita ingresar, consultar y ordenar las notas de hasta quince estudiantes, con el fin de comprender el funcionamiento de instrucciones tales como la impresión en pantalla, lectura I/O, y operaciones matemáticas básicas. Este se enmarca entonces dentro del paradigma de programación imperativa y bajo nivel lo que permitirá reforzar conceptos fundamentales de organización de memoria, gestión de registros, manipulación de pila y control de flujo mediante saltos, todos esenciales para la comprensión del funcionamiento de las computadoras a nivel de hardware.

2. Descripción de los algoritmos de solución

El sistema RegistroCE es una aplicación en lenguaje ensamblador que gestiona el registro y análisis de calificaciones de estudiantes. Permite ingresar, buscar, ordenar y analizar estadísticas de hasta 15 estudiantes con notas decimales de precisión.

Algoritmos de solución Desarrollados

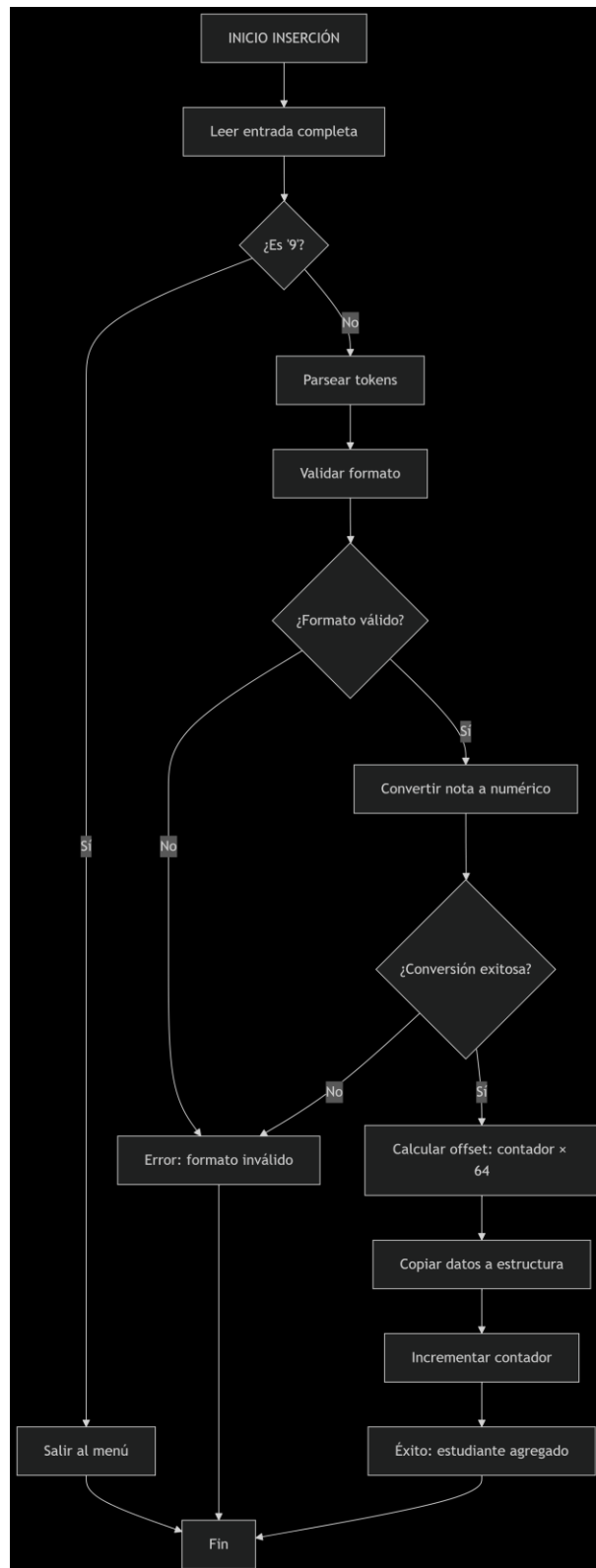
1. Algoritmo de Inserción y Parseo de Datos

- a. **Descripción:** El sistema implementa un algoritmo de inserción que procesa y valida la entrada de nuevos estudiantes. La particularidad es que convierte las notas decimales a formato numérico fija (x10000) y realiza validación de formato antes de almacenar los datos.

b. Justificación

- **Validación temprana:** Detecta errores antes de modificar datos.
- **Conversión robusta:** Manejo específico de notas decimales.
- **Eficiencia $O(1)$:** Inserción directa por cálculo de offset.

c. Diagrama de flujo



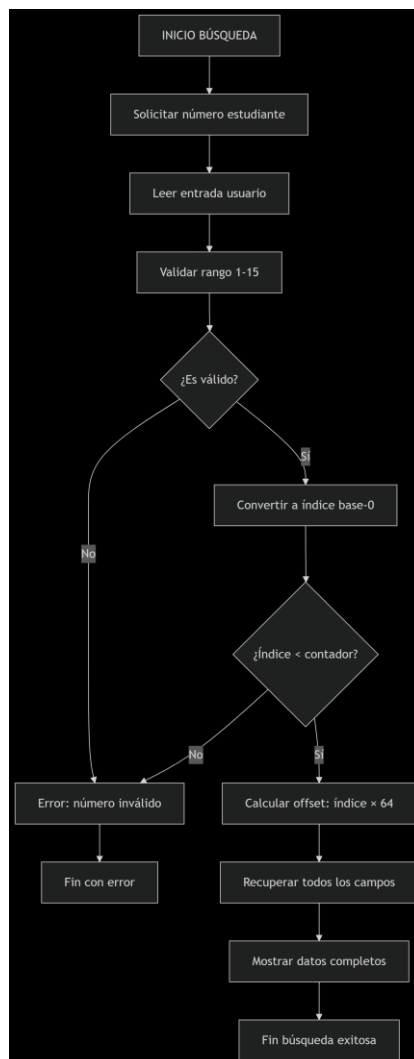
2. Algoritmo de búsqueda por índice

a. **Descripción:** El sistema implementa un algoritmo de búsqueda directa por índice para recuperar estudiantes. La particularidad es que utiliza cálculo matemático de posiciones para acceso inmediato sin necesidad de búsqueda secuencial, mostrando todos los campos del estudiante.

b. Justificación

- **Acceso directo $O(1)$:** Cálculo matemático de posición.
- **Validación completa:** Verifica existencia y validez.
- **Eficiencia máxima:** Sin búsquedas secuenciales.
- **Recuperación completa:** Todos los campos en una operación.

c. Diagrama de flujo



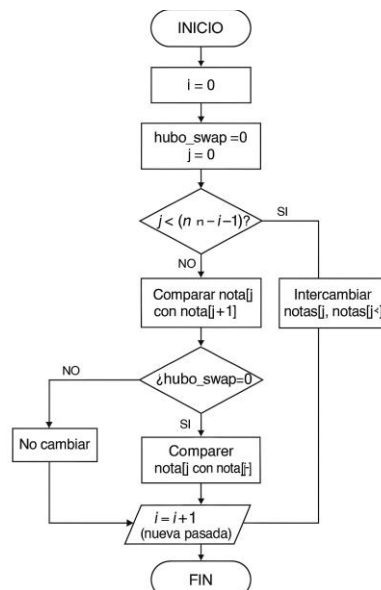
3. Algoritmo de Ordenamiento – Bubble Sort

a. **Descripción:** El sistema implementa un algoritmo Bubble Sort optimizado para ordenar las calificaciones de los estudiantes. La particularidad es que ordena un array de índices en lugar de mover físicamente los datos de los estudiantes.

b. Justificación

- **Optimización de memoria:** Ordena índices, no datos
- **Eficiencia aceptable:** Para $n=15$, máximo 105
- **Estabilidad:** Mantiene el orden relativo de elementos iguales

c. Diagrama de Flujo



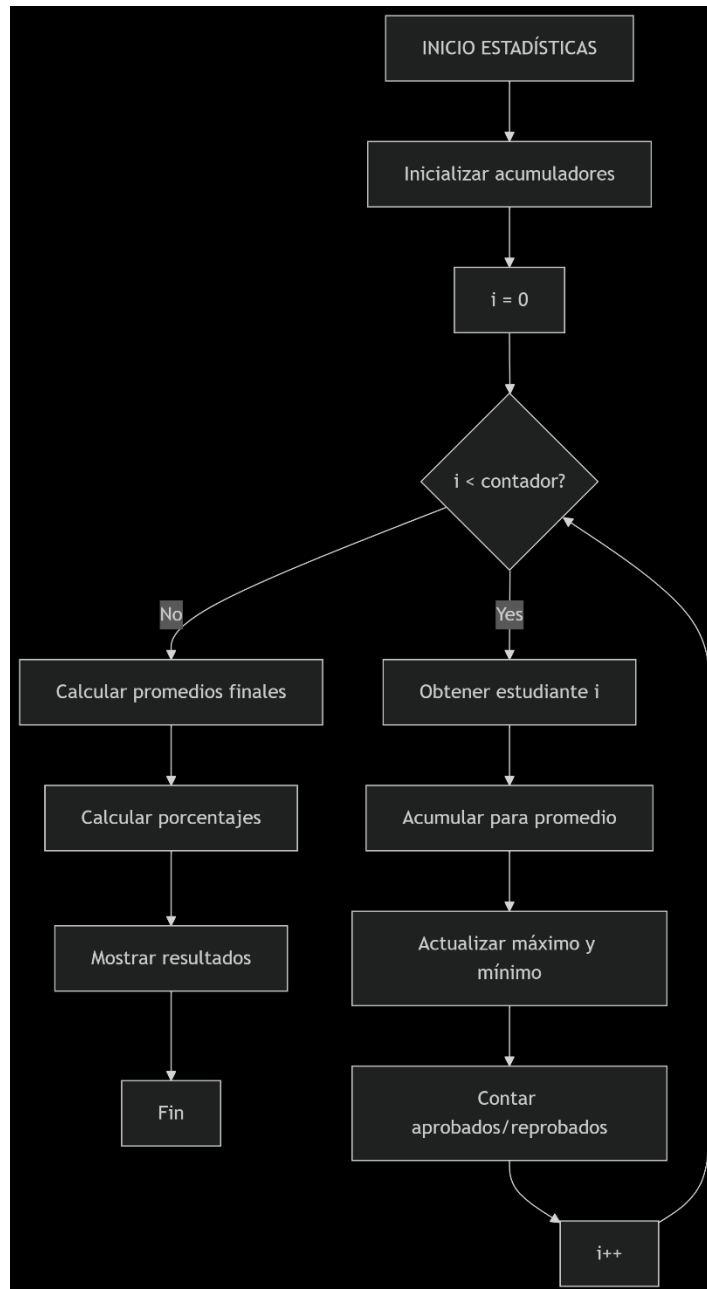
4. Algoritmo de Cálculo de Estadísticas

a. **Descripción:** El sistema implementa un algoritmo de cálculo de estadísticas que procesa todos los datos académicos. La particularidad es que realiza una sola pasada sobre los datos para calcular múltiples métricas (promedio, máximo, mínimo, aprobados) simultáneamente, optimizando el rendimiento.

b. Justificación

- **Eficiencia $O(n)$:** Todos los cálculos en una sola iteración
- **Precisión decimal:** Manejo matemático correcto
- **Actualización en tiempo real:** Máximo y mínimo se actualizan durante la pasada.
- **Optimización:** Mínimo uso de operaciones costosas.

c. Diagrama de flujo



3. Descripción de los procedimientos y funciones

Se trabajarán las subrutinas como grupos de tal forma que su descripción sea más detallada y fácil de leer.

Grupo 1: Núcleo del Programa y Gestión del Menú

Es el esqueleto principal del programa, responsable de la interacción inicial con el usuario y la coordinación de las funcionalidades del sistema.

1. Procedimiento: MAIN

- **Propósito:** Punto de entrada principal y ciclo central del programa. Su función es inicializar el segmento de datos y presentar continuamente el menú de opciones al usuario, redirigiendo el flujo de ejecución según la selección realizada.
- **Entrada:** Ninguna.
- **Salida:** Ninguna directa. Controla la ejecución de todo el programa.
- **Descripción:** Tras inicializar el registro DS con el segmento de datos, entra en un ciclo infinito etiquetado como INICIO. En cada iteración, utiliza la interrupción INT 21h con la función AH=09h para desplegar en pantalla todas las cadenas que conforman el menú principal. Posteriormente, con la función AH=01h de la misma interrupción, captura un único carácter de la entrada estándar. Mediante una serie de comparaciones (CMP AL, 'X') y saltos condicionales (JE OPCIONX), transfiere el control a los procedimientos específicos de cada opción. La opción '5' termina el programa mediante INT 21h con AH=4Ch.

2. Procedimiento: OPCION1

- **Propósito:** Gestiona la lógica para ingresar nuevos registros de estudiantes al sistema.
- **Entrada:** Ninguna. Lee la variable global contador.
- **Salida:** Modifica el arreglo estudiantes e incrementa la variable global contador si el ingreso es exitoso.
- **Descripción:** Primero verifica si hay espacio disponible en el array (CMP contador, 15). Si la lista está llena, muestra un mensaje de error y retorna al menú. De lo contrario, solicita al usuario que ingrese los datos en el formato especificado. Delega la lectura de la entrada a LEER_ENTRADA_COMPLETA y su procesamiento a PARSE_ENTRADA.

Actúa en consecuencia al estado del flag de carry (JC) devuelto por estas subrutinas, mostrando mensajes de éxito o error.

3. Procedimiento: OPCION3

- **Propósito:** Coordina el proceso de búsqueda por índice y visualización de un estudiante específico.
- **Entrada:** Ninguna.
- **Salida:** Muestra datos en pantalla.
- **Descripción:** Ejecuta un llamado a la subrutina BUSCAR_ESTUDIANTE, la cual se encarga de toda la lógica de esta funcionalidad.

4. Procedimiento: OPCION2 y OPCION4

- **Propósito:** Placeholders para las funcionalidades de mostrar estadísticas y ordenar calificaciones, respectivamente.
- **Descripción:** En el estado actual del programa, estos procedimientos se limitan a mostrar un mensaje en pantalla indicando la opción seleccionada (M21, M44, M45) antes de retornar al menú principal. Su implementación completa está pendiente.

Grupo 2: Procesamiento y Tokenización de Entrada

Este grupo se encarga de la interacción de bajo nivel con el teclado y del procesamiento de la cadena de caracteres ingresada por el usuario, dividiéndola en los tokens que conforman los datos de un estudiante.

1. Procedimiento: LEER_ENTRADA_COMPLETA

- **Propósito:** Lee una secuencia de caracteres desde la entrada estándar hasta que se presione la tecla Enter o se alcance el límite del buffer.
- **Entrada:** DI debe apuntar al inicio del buffer de destino.
- **Salida:** El buffer contiene la cadena ingresada, terminada con el carácter '\$'. El flag de carry (CF) se activa si el usuario ingresa únicamente "9" seguido de Enter, indicando que quiere salir.
- **Descripción:** Utiliza un ciclo con INT 21h, AH=01h para leer caracteres de uno en uno. Cada carácter se almacena en la posición apuntada por DI, incrementando el puntero. El ciclo se rompe al detectar el carácter de retorno de carro (ASCII 13). Incluye una verificación especial para la secuencia de salida ("9" + Enter).

2. Procedimiento: PARSE_ENTRADA

- **Propósito:** Procesa la cadena en el buffer, los divide en tokens (nombre, apellido1, apellido2, nota) y los almacena en la estructura de estudiante correspondiente.
- **Entrada:** SI apunta a la posición en el array estudiantes donde se guardarán los datos.
- **Salida:** Los campos de la estructura del estudiante están poblados con los tokens. El flag de carry (CF) se activa si ocurre un error de formato durante el parsing o la conversión.
- **Descripción:** Este procedimiento actúa como un coordinador. En secuencia, llama a leer_token para obtener cada uno de los cuatro tokens esperados. Para los primeros tres (nombre, apellidos), llama a sus respectivas subrutinas de copia (copiar_nombre, etc.). Para el último token (nota), llama a convertir_nota. Si cualquier de estas subrutinas falla, activa el CF para indicar error.

3. Procedimiento: leer_token

- **Propósito:** Extrae un único token desde la posición actual del buffer de entrada, avanzando el puntero más allá del token y su delimitador.
- **Entrada:** DI apunta a la posición actual en el buffer.
- **Salida:** El token extraído se almacena en temp_nota. DI avanza hasta la posición inmediatamente después del delimitador (espacio o fin de cadena).
- **Descripción:** Salta cualquier espacio inicial. Luego, lee carácter por carácter, copiándolos a temp_nota, hasta encontrar un espacio, un retorno de carro o el fin de la cadena ('\$'). Finaliza la cadena en temp_nota con '\$'.

4. Procedimiento: saltar_espacios_sin_avanzar (y leer_token_sin_avanzar)

- **Propósito:** saltar_espacios_sin_avanzar ajusta el puntero DI para que apunte al primer carácter no espacial de la cadena. leer_token_sin_avanzar es una variante de leer_token que no avanza el puntero principal.
- **Descripción:** Si bien están implementadas, estas rutinas tienen un uso limitado en la versión actual del parser, el cual está principalmente basado en leer_token.

Grupo 3: Manipulación de Estructuras y Conversión de Datos

Este grupo de procedimientos es el responsable de tomar los tokens procesados y almacenarlos en las estructuras de datos en memoria, así como de realizar las conversiones necesarias entre formatos como por ejemplo ASCII a binario.

1. Procedimiento: copiar_nombre

- **Propósito:** Copiar el contenido del buffer temporal temp_nota al campo nombre de la estructura del estudiante en memoria.
- **Entrada:** SI apunta al inicio de la estructura de un estudiante en el array estudiantes. El token a copiar debe estar en temp_nota.
- **Salida:** El campo nombre (offset NOMBRE_OFFSET) de la estructura del estudiante es poblado con los caracteres del token, seguido de un carácter terminador '\$'.
- **Descripción:** Utiliza un puntero origen (DI) apuntando a temp_nota y un puntero destino (calculado como SI + NOMBRE_OFFSET + CX). Mediante un ciclo, copia carácter por carácter hasta encontrar el terminador '\$' en el token. Asegura que el campo en la estructura también termine con '\$'.

2. Procedimiento: copiar_apellido1

- **Propósito:** Idéntico a copiar_nombre, pero dirige el token hacia el campo apellido1 de la estructura.
- **Entrada:** SI apunta a la estructura del estudiante. Token en temp_nota.
- **Salida:** El campo apellido1 (offset APELLIDO1_OFFSET) es poblado.
- **Descripción:** Misma lógica que copiar_nombre, pero calcula el destino como SI + APELLIDO1_OFFSET + CX.

3. Procedimiento: copiar_apellido2

- **Propósito:** Idéntico a los anteriores, pero copia el token al campo apellido2.
- **Entrada:** SI apunta a la estructura del estudiante. Token en temp_nota.
- **Salida:** El campo apellido2 (offset APELLIDO2_OFFSET) es poblado.
- **Descripción:** Misma lógica, destino en SI + APELLIDO2_OFFSET + CX.

4. Procedimiento: convertir_nota

- **Propósito:** Convertir la representación en ASCII de la nota (almacenada en temp_nota) en un valor numérico binario y almacenarlo en la estructura.

- **Entrada:** SI apunta a la estructura del estudiante. El token a convertir debe estar en temp_notas.
- **Salida:** El campo nota (offset NOTA_OFFSET) de la estructura es poblado con el valor convertido. El flag de carry (CF) se activa si se encuentra un carácter no numérico o un formato inválido durante la conversión.
- **Descripción:** Implementa un algoritmo de conversión de ASCII a binario para integers. Recorre el string en temp_notas, verifica que cada carácter sea un dígito ('0'-'9'), convierte cada dígito a su valor binario y construye el número final mediante multiplicaciones por 10 y sumas sucesivas. El resultado final se almacena en la palabra de memoria [SI + NOTA_OFFSET]. *Nota: En la versión actual, este procedimiento no soporta números flotantes; esta funcionalidad fue planeada pero no completada.*

Grupo 4: Salida, Visualización y Lógica de Búsqueda

Este grupo de procedimientos maneja la recuperación de datos desde las estructuras en memoria y la presentación en pantalla al usuario.

1. Procedimiento: BUSCAR_ESTUDIANTE

- **Propósito:** Busca un estudiante en el array basado en su número de índice (1-15) y mostrar toda su información en pantalla.
- **Entrada:** Solicita el número de estudiante al usuario mediante una entrada estandar.
- **Salida:** Muestra los datos del estudiante en pantalla o un mensaje de error.
- **Descripción:** Este procedimiento coordina la búsqueda. Primero muestra un prompt y lee el número ingresado por el usuario. Valida que esté en el rango 1-15 y que no exceda el contador de estudiantes ingresados. Si es válido, calcula la posición exacta en el array: $\text{Posición} = (\text{Número} - 1) * \text{ESTUDIANTE_SIZE}$. Luego, pasa este puntero (SI) a MOSTRAR_ESTUDIANTE_COMPLETO para que muestre los datos. Maneja los casos de error con mensajes apropiados.

2. Procedimiento: MOSTRAR_ESTUDIANTE_COMPLETO

- **Propósito:** Recibe un puntero a una estructura de estudiante y muestra todos sus campos (nombre, apellidos, nota) de forma formateada en pantalla.

- **Entrada:** SI apunta a la estructura de un estudiante en memoria.
- **Salida:** Visualiza la información en pantalla.
- **Descripción:** Calcula los offsets para cada campo (NOMBRE_OFFSET, APELLIDO1_OFFSET, etc.) y utiliza la interrupción INT 21h, AH=09h para imprimir las cadenas de nombre y apellidos. Para la nota, carga el valor binario desde [SI + NOTA_OFFSET] en BX y llama a MOSTRAR_NUMERO.

3. Procedimiento: MOSTRAR_NUMERO

- **Propósito:** Convierte un valor numérico binario de 16 bits a su representación decimal en ASCII y lo muestra en pantalla.
- **Entrada:** BX contiene el número a mostrar.
- **Salida:** Muestra el número.
- **Descripción:** Implementa el algoritmo clásico de división sucesiva por 10. El número en AX (copia de BX) se divide repetidamente por 10. Los restos de cada división (que representan los dígitos) se guardan en la pila. Luego, se extraen de la pila en orden inverso (del más significativo al menos significativo), se convierten a ASCII (sumando '0') y se imprimen usando INT 21h, AH=02h.

4. Procedimiento: MOSTRAR_NUMERO_1DIGITO

- **Propósito:** Versión optimizada de MOSTRAR_NUMERO para números de un solo dígito (0-9).
- **Entrada:** AL contiene el dígito a mostrar.
- **Salida:** Muestra un solo dígito en pantalla.
- **Descripción:** Convierte el valor en AL a ASCII sumando el carácter '0' y lo imprime directamente usando INT 21h, AH=02h. Es más eficiente que MOSTRAR_NUMERO para este caso específico.

4. Descripción de las estructuras de datos

Para poder almacenar los datos de los estudiantes se reserva, en la sección de .DATA, un bloque contiguo de memoria que simula una lista de registros, nodos, de tamaño fijo. Cada registro representa a un estudiante y contiene campos para nombre, primer apellido, segundo apellido y nota, la cual se separa en parte entera y parte decimal.

Se definieron constantes de desplazamiento (offsets) que indican, en bytes, la posición de cada campo respecto a la dirección base del registro. El tamaño total del registro es 64 bytes. La parte decimal se guarda escalada en un entero sin signo, permitiendo como máximo 4 decimales.

- NOMBRE_OFFSET = 0
- APELLIDO1_OFFSET = 20
- APELLIDO2_OFFSET = 40
- NOTA_ENTERA_OFFSET 60
- NOTA_DECIMAL_OFFSET 62

Para manejar el arreglo de estudiantes se utilizó un buffer que reserva (15 * tamaño estudiante) bytes para almacenar 15 registros continuos. La variable contador mantiene cuantos registros han sido ocupados. En el caso del bubblesort se creo otro arreglo de 15 bytes para almacenar los índices de los estudiantes y solo organizar estos en vez de mover todo el bloque de 64 bytes.

Cuando se quiere acceder a un estudiante, o bien agregar uno, se toma la posición inicial de “estudiantes” y se le suma la multiplicación del contador por el tamaño total del nodo, de esta forma permite colocar el registro en la dirección de memoria que se desea leer o escribir. Para obtener o escribir un atributo específico se utilizan las variables offset donde una vez obtenida la dirección inicial del nodo deseado se suma el offset correspondiente a la posición de inicio.

5. Problemas encontrados

• Perdida de valores en registros

Durante el desarrollo del programa se encontraron problemas donde los resultados obtenidos no eran los esperados, lo que llevaba a realizar corridas manuales para identificar la fuente del error. En la mayoría de los casos se debía a operaciones como CMP o MUL que modificaban los registros que se estaban utilizando.

Para solucionar este problema se utilizo el Stack, el cual permite guardar registros en una pila para posteriormente recuperarlo, mediante los comandos PUSH y POP. Esto fue de suma importancia con los procedimientos pues en ocasiones el valor que se necesitaba estaba dentro de un procedimiento, pero al volver al flujo principal se perdía.

- **Preservaciónn de banderas**

Para poder controlar la acción que el sistema debe tomar según las entradas del usuario se utilizan los comandos JUMP, y sus variaciones lógicas. En caso de que alguna operación modificara las banderas antes del momento que se ocupaban para tomar decisiones el resultado no era el esperado. Para solucionar esto, similar al caso anterior, se utilizo la pila para guardar el estado de las banderas mediante el comando PUSHF y POPF.

- **Desbordamiento por restricción de 16 bits**

Los requerimientos solicitaban el manejo de 5 decimales, sin embargo, la arquitectura 8086 cuenta con registros de hasta 16 bits, lo cual en decimal permite almacenar números hasta el 65536, de forma que si se ingresa 99.99999 el resultado esperado no es correcto porque solo puede almacenar los 16 bits y lo demás había que guardarlo en otro registro.

Inicialmente se implementó el programa para soportar 2 decimales, se presentaron problemas para adaptar la lógica a 5 decimales y por restricciones de tiempo se dejó en 4 decimales para poder dedicar tiempo al resto de funcionalidades por lo que este problema no se pudo solucionar dentro del tiempo de entrega.

- **Accesos a memoria**

Los accesos a memoria, para leer o escribir, fueron uno de los problemas iniciales, donde la mayor problemática era diferencias entre dirección de memoria y el contenido de esta. Esto generaba problemas de direccionamiento al no pasar los valores correctos o no obtener el resultado esperado, pues se guardaba la dirección y no el contenido. El uso de OFFSET fue clave para solucionar estos problemas.

- **Lectura y separación de entradas de usuario**

Para moverse dentro del menú solo se solicita un carácter al usuario, redireccionando según el valor ingresado. Para agregar a un estudiante y su nota es diferente, se debe esperar hasta que el usuario finalice presionando ENTER lo cual activa la lógica de validación y guardado de los datos. Se encontraron problemas pues el usuario puede ingresar 9 al inicio del mensaje para cancelar agregar un estudiante, esto se logró implementar, pero en caso de agregar un 9 dentro de la nota generaba problemas y cancelaba la función de agregar.

Para solucionar este problema se tuvo que reestructurar el código de forma que el usuario pudiera usar libremente el 9 a la hora de agregar la nota de un estudiante.

6. Plan de actividades realizadas

Actividad	Descripción	Tiempo Estimado	Responsable	Entrega
Análisis inicial	Analizar el documento, comprender que se requiere, trazar plan.	3 horas	Todos	21 de agosto
Core del Sistema	Definir la estructura, programar el menú principal, control de input del usuario y control de saltos a cada opción.	4 horas	Luis	24 de agosto
Modulo Entrada Datos	Implementar la lectura del input del usuario (LEER_ENTRADA_COMPLETA) y las validaciones de formato.	5 horas	Saddy	25 de agosto
Diseño Parser entrada	Implementar la lógica de recorrido del buffer de entrada, volver token los datos y guardarlos en la estructura de estudiante en memoria.	6 horas	Luis	27 de agosto
Módulo de Búsqueda	Desarrollar el algoritmo para calcular la posición en memoria de un estudiante basado en su índice y mostrar sus datos.	4 horas	Saddy	28 de agosto
Manejo de Decimales	Adaptar las rutinas de conversión y visualización para soportar 2 decimales en las notas	4 horas	Luis	29 de agosto
Manejo de Decimales	Investigar e implementar la aritmética necesaria para aumentar la precisión a 4 y 5 decimales. Resolver problemas de precisión y representación.	8 horas	Luis	31 de agosto
Módulo de Estadísticas	Implementar los algoritmos para calcular el promedio, nota máxima, mínima, porcentaje y aprobados/reprobados	6 horas	Gabriel	2 de septiembre
Algoritmo de ordenamiento	Implementar y depurar el algoritmo de ordenamiento (burbuja/selección) para ordenar el arreglo de estudiantes basado en su nota	7 horas	Gabriel	3 de septiembre
Integración, depuración final y pruebas	Integrar todos los módulos, realizar las pruebas exhaustivas de todas las funciones corregir errores encontrados	6 horas	Todos	4 de septiembre
Elaboración Documentación técnica	Redactar la documentación, describir algoritmos, estructuras y decisiones de diseño.	4h	Saddy	4 de septiembre

Actividad	Descripción	Tiempo Estimado	Responsable	Entrega
Elaboración manual de usuario	Redactar una guía clara para el usuario	2h	Saddy	4 de septiembre
Preparación para la defensa	Preparar una presentación 2-3 diapositivas y organizar la demostración	2h	Saddy	4 de septiembre

7. Conclusiones

El desarrollo del sistema RegistroCE en lenguaje ensamblador 8086 resulto ser un desafío considerable que permitió aprender y poner en práctica los conceptos fundamentales de la programación de bajo nivel tales como la gestión manual de memoria, el uso de registros para cálculos de direcciones, manipulación de la pila para el paso de parámetros en subrutinas y el control preciso del flujo del programa mediante instrucciones de salto condicional e incondicional. Se logro cumplir con la mayoría de los objetivos de este, como implementar un sistema interactivo con un menú principal funcional con la capacidad de ingresar datos en un formato, almacenarlos en estructuras en memoria y acceder a ellos mediante la búsqueda por índice. La decisión de estructurar el código en subrutinas modulares como LEER_ENTRADA_COMPLETA, PARSE_ENTRADA y BUSCAR_ESTUDIANTE fue importante para abarcar todo el proyecto, facilitar la depuración y permitir que varios miembros del equipo trabajaran en partes diferentes del sistema al mismo tiempo. Sin embargo, se encontraron complicaciones en la manipulación de datos como en la implementación del parser y la mayor complicación fue la duración en el desarrollo en el manejo de decimales, no se logró alcanzar los 5 decimales. Se planificaron funcionalidades avanzadas como el ordenamiento y estadísticas, pero por la complejidad misma del ASM y la poca experiencia en ensamblador llevo a que estas tareas tomaran más tiempo del estimado, demostrando lo complicado en hacer estimaciones precisas en programación de bajo nivel

8. Recomendaciones

1. Investigar bibliotecas o rutinas existentes: Para operaciones complejas como la aritmética de punto flotante se recomienda buscar e integrar rutinas ya existentes y probadas en vez de implementarlas desde cero, esto permitirá ahorrar tiempo y reduce drásticamente la probabilidad de errores.
2. Definir un MVP y priorizarlo: En proyectos con una fecha de entrega, se recomienda enfocar todos los esfuerzos en un mínimo producto viable que cumpla las funcionales centrales como ingreso, almacenamiento y consulta antes de desarrollar características secundarias como las estadísticas o el ordenamiento.
3. Documentar en paralelo al desarrollo: Llevar una documentación técnica y una bitácora detallada durante el proceso de código facilita enormemente la integración del trabajo entre miembros del equipo y hace que al final la elaboración del documento sea más eficiente y precisa.
4. Gestionar los riesgos técnicos: Identificar desde un inicio que componentes del proyecto presentan mayor riesgo, aquellos que parecen muy complejos y dedicarle a estos mayor recursos y tiempo, asumiendo que surgirán complicaciones.

9. Bibliografía

Peinador, J. F., & Ruiz, D. S. (1998). ENSAMBLADOR DEL 8086/88.

Acevedo, M. J. [NEOMATRIX]. (2020, mayo 10). *Curso ensamblador (2020)* [Video]. YouTube.

https://www.youtube.com/watch?v=oLsk9J_mViE&list=PLZw5VfkTcc8Mzz6HS6-XNxfnEyHdyTlmP&index=1

DeepSeek. (2025). *DeepSeek Chat* (Versión V3) [Modelo de lenguaje grande]. <https://chat.deepseek.com>

OpenAI. (2025). *ChatGPT* (Versión GPT-4) [Modelo de lenguaje grande]. <https://chat.openai.com>

10. Bitácora

Fecha: 21/08/2025

Participantes: Luis, Saddy, Gabriel

Objetivo: Análisis inicial y planificación del proyecto.

Actividades realizadas:

- Reunión inicial por Discord para analizar el documento de la tarea.
- Discutimos los requerimientos mínimos y definimos las estructuras de datos necesarias.
- Decidimos usar una estructura de 64 bytes por estudiante: 20 bytes para nombre, 20 para cada apellido y 4 para la nota.
- Trazamos un plan inicial de trabajo y dividimos las tareas. Luis se encargará del núcleo y el parser, Saddy de la entrada y búsqueda, y Gabriel de las estadísticas y ordenamiento.

- Investigamos en documentaciones y en videos de YouTube sobre interrupciones DOS (INT 21h) para entrada/salida.

Problemas encontrados: Duda sobre cómo manejar números flotantes en ensamblador. No hay soporte hardware.

Decisiones tomadas: Empezar solo trabajando con enteros. Dejar los decimales para el futuro después de investigar más.

Plan para la próxima sesión: Luis comenzará a codificar el menú principal.

Fecha: 24/08/2025

Participantes: Luis

Objetivo: Implementar el núcleo del sistema (menú principal y control de flujo).

Actividades realizadas:

- Se definieron todos los mensajes del menú en el segmento .DATA (MI1, MI2, MI3, etc)
- Se codificó la etiqueta INICIO: que muestra el menú usando MOV AH, 09h e INT 21h.
- Se implementó la lectura de la opción del usuario con MOV AH, 01h e INT 21h.
- Se codificaron las comparaciones (CMP AL, '1') y los saltos (JE OPCION1) para cada opción del menú.

- Se probó que el flujo funcione correctamente, saltando a etiquetas temporales que solo muestren un mensaje de confirmación, ejemplo: "Opción 1 seleccionada"
Problemas encontrados: Ninguno crítico. El flujo básico funciona.
Plan para la próxima sesión: Saddy implementará la rutina de lectura de entrada para la Opción 1.

Fecha: 25/08/2025

Participantes: Saddy

Objetivo: Implementar la lectura de entrada del usuario para la Opción 1.

Actividades realizadas:

- Se definieron los mensajes específicos para la opción 1 (M11, M13, M15) en .DATA.
- Se codificó la rutina LEER_ENTRADA_COMPLETA. Lee caracteres hasta presionar ENTER (ASCII 13) o llenar el buffer (99 caracteres).
- Se agregó una validación especial: si el usuario ingresa solo "9" y ENTER, se activa el flag de carry para salir al menú.
- Se implementó la validación básica en OPCION1 que verifica que no se excedan 15 estudiantes (CMP contador, 15).

Problemas encontrados: La rutina lee bien, pero aún no procesa la entrada.

Decisiones tomadas: La validación del formato (4 tokens) se hará en el parser, no en el lector.

Plan para la próxima sesión: Luis comenzará con el parser (PARSE_ENTRADA).

Fecha: 27/08/2025

Participantes: Luis

Objetivo: Implementar el parser de entrada (PARSE_ENTRADA).

Actividades realizadas:

- Se codificó la rutina leer_token que recorre el buffer, salta espacios y extrae tokens hasta encontrar un espacio o el fin de cadena.
- Se implementaron las subrutinas copiar_nombre, copiar_apellido1 y copiar_apellido2. Usan el offset de la estructura (SI) y un contador (CX) para copiar cada carácter del token al lugar correcto en memoria.
- Se calcula la posición del estudiante en el array: MOV AL, contador MOV BL, ESTUDIANTE_SIZE MUL BL ADD SI, AX.

- Se hizo una prueba exitosa. Los strings se guardaron correctamente.
Problemas encontrados: La rutina convertir nota solo convierte enteros. Los decimales crashean el programa.
Decisiones tomadas: trabajar con enteros por ahora para poder seguir integrando. Gabriel se encargará de la conversión a decimal después.
Plan para la próxima sesión: Saddy implementará la búsqueda por índice (Opción 3).

Fecha: 28/08/2025

Participantes: Saddy

Objetivo: Implementar la búsqueda de estudiante por índice (Opción 3).

Actividades realizadas:

- Se codificó la rutina BUSCAR_ESTUDIANTE. Pide el número, valida que esté entre 1-15 y que no exceda el contador.
- Se implementa el cálculo de posición: MOV AL, CL DEC CL MOV BL, ESTUDIANTE_SIZE MUL BL ADD SI, AX.
- Se creó MOSTRAR_ESTUDIANTE_COMPLETO para desplegar todos los datos recuperados de memoria.
- Se probó con el primer estudiante ingresado y mostró correctamente.
Problemas encontrados: La nota se muestra como un entero. Si se agregan decimales muestra información errónea.
Decisiones tomadas: Es funcional para enteros. Se ajustará luego para decimales.
Plan para la próxima sesión: Luis investigará cómo implementar el manejo de decimales.

Fecha: 29/08/2025

Participantes: Luis

Objetivo: Investigar e implementar manejo de 2 decimales.

Actividades realizadas:

- Se investigó y Se decidió usar aritmética de punto fijo: multiplicar por 100 para guardar la nota como un entero que representa centésimas.
- Se modificó convertir_nota para detectar el punto decimal. Ahora convierte la parte entera y la decimal por separado y las combina.
- Se adaptó MOSTRAR_NUMERO para dividir el valor entre 100 e imprimir el punto y los dos decimales.

- Se probó con "90.50". Se guarda como 9050 en memoria y se muestra correctamente.
Problemas encontrados: La lógica es podría fallar facilmente. Si el usuario no usa exactamente 2 decimales crashea.
Decisiones tomadas: Es un buen avance. Mañana se intentará generalizar para más decimales.
Plan para la próxima sesión: Generalizar la solución para 4 o 5 decimales.

Fecha: 30/08/2025

Participantes: Luis, Gabriel

Objetivo: Implementar soporte para 4-5 decimales.

Actividades realizadas:

- Intentamos multiplicar por 100000 para guardar 5 decimales. Resulto en Desbordamiento $100.00000 * 100000 = 10,000,000$, que supera los 65,535 (límite de 16 bits).
- Probamos con 4 decimales ($\times 10000$). $100.0000 * 10000 = 1,000,000$. Sigue desbordando un registro de 16 bits.
- Decidimos usar 2 registros (DX:AX) para operaciones de 32 bits, pero es muy complejo para el tiempo restante.
- Consultamos a ChatGPT para ideas sobre aritmética de punto fijo con números grandes. Sugirió usar una representación de 32 bits, pero implica reescribir mucha lógica.

Problemas encontrados: Desbordamiento de registros. Complejidad algorítmica.

Decisiones tomadas: Priorizar la funcionalidad sobre la precisión. Nos quedamos con 2 decimales que funcionan bien y son suficientes para el caso de uso.

Plan para la próxima sesión: Gabriel comenzará con el módulo de estadísticas.

Fecha: 02/09/2025

Participantes: Gabriel

Objetivo: Implementar el módulo de estadísticas (Opción 2).

Actividades realizadas:

- La opción 2 solo muestra el mensaje "Aquí se muestran las estadísticas". Es un placeholder.

- Se comenzó a diseñar la lógica para recorrer el array de estudiantes, sumar las notas y calcular el promedio.
- Se investigó cómo hacer división con DIV para el cálculo del promedio.
Problemas encontrados: Las notas se guardan como enteros que representan centésimas (x100). Hay que ajustar todos los cálculos para esto.
Decisiones tomadas: Las estadísticas y el ordenamiento son las siguientes prioridades.
Plan para la próxima sesión: Reunión de emergencia para definir si alcanzamos a implementar todo.

Fecha: 03/09/2025 - Reunión de Emergencia

Participantes: Luis, Saddy, Gabriel

Objetivo: Reevaluar el plan de trabajo final frente a la fecha de entrega.

Actividades realizadas:

- Revisión del estado actual: El sistema ingresa, guarda, busca y muestra estudiantes con 2 decimales. Faltan estadísticas y ordenamiento.
- Evaluamos el tiempo que nos queda vs. la complejidad de implementar y depurar los algoritmos restantes.
- Decidimos que Saddy se enfoque al 100% en la documentación técnica para entregarla completa.
- Gabriel intentará implementar un algoritmo de burbuja muy básico para ordenamiento.
- Luis hará una rutina simple de promedio para las estadísticas.
Decisiones tomadas: Entregar un sistema funcional con las opciones 1 y 3 completas, y las 2 y 4 de manera básica o placeholder. La documentación debe ser excelente y explicar este trade-off.

Plan para la próxima sesión: Marcha forzada. Codificación final y documentación.

Enlace Repositorio de github:

<https://github.com/SaddyGR3/RegistroCE.git>

